

NORMA 220501096

Documento técnico de código fuente

Cooperativa Financiera El Progreso

Estructura, flujo, tecnologías y decisiones técnicas
Versión 1.0 | 2026

1. Estructura del proyecto

```
CooperativaElProgreso/  
├─ Enums/           Tipos enumerados  
├─ Models/          Entidades y resultados  
│   └─ Reports/     DTO de reportes  
├─ IInterfaces/     Contratos de servicios y repositorios  
├─ Repositories/    Persistencia en memoria  
├─ Services/        Reglas del negocio  
├─ Infrastructure/ExternalServices/ Cliente de TRM  
├─ Data/            Almacenamiento y datos iniciales  
└─ UI/             Menús de consola  
  
CooperativaElProgreso.Tests/    Pruebas unitarias xUnit
```

2. Descripción de carpetas y módulos

Carpeta	Descripción
Models	Clases <code>Member</code> , <code>Movement</code> , <code>ExchangeRate</code> y resultados genéricos.
IInterfaces	Contratos que desacoplan las capas.
Repositories	Operaciones sobre el almacenamiento en memoria.
Services	Validaciones, movimientos, saldo y reportes.
Infrastructure	Comunicación con la API externa de TRM.
Data	Listas compartidas y datos de demostración.
UI	Menús y navegación de consola.
Tests	Pruebas de reglas de negocio y dobles de prueba.

3. Flujo general del sistema

- `Program.cs` crea el almacenamiento, repositorios, servicios y menús.
- El usuario selecciona el menú de cajero o de administración.
- El menú llama una interfaz de servicio.
- El servicio valida la entrada y aplica reglas del negocio.
- El repositorio consulta o agrega información al almacenamiento.
- El servicio retorna un `OperationResult` y la UI muestra el mensaje.

4. Flujo de consulta de TRM

1. `MemberService.GetBalanceInUsdAsync` valida que exista el asociado.
2. Solicita la TRM a `IExchangeRateService`.
3. `TrmExchangeRateService` ejecuta un `GET` asíncrono con `HttpClient`.
4. Valida HTTP, JSON, valor y fechas.
5. Calcula el saldo en dólares y devuelve el resultado.

5. Fragmentos relevantes

Composición de dependencias

```
IMemberRepository memberRepository = new MemberRepository(store);
IMovementRepository movementRepository = new MovementRepository(store);
IExchangeRateService exchangeRateService = new TrmExchangeRateService();
IMemberService memberService = new MemberService(
    memberRepository, movementRepository, exchangeRateService);
```

El programa inyecta implementaciones concretas detrás de interfaces. Esto facilita reemplazarlas en pruebas o en una futura implementación.

Saldo derivado

```
public decimal GetBalance(Guid memberId) =>
    _movementRepository.GetByMemberId(memberId)
        .Sum(m => m.SignedTotal);
```

El saldo se calcula con el historial, en lugar de almacenarse como un campo independiente.

Regla de retiro

```
var fee = amount > 1_000_000m ? 8_000m : 0m;
var currentBalance = movements.Sum(m => m.SignedTotal);
var totalToDeduct = amount + fee;
if (totalToDeduct > currentBalance)
    return OperationResult<Movement>.Fail("Saldo insuficiente");
```

La validación se realiza antes de guardar el movimiento; por tanto, una operación rechazada no altera el historial.

Integración tolerante a fallos

```
using var response = await _httpClient.GetAsync(RequestUri);
var body = await response.Content.ReadAsStringAsync();
using var document = JsonDocument.Parse(body);
```

Las excepciones de red, cancelación, JSON y formato se convierten en un resultado fallido controlado.

6. Tecnologías utilizadas

- C# 14.
- .NET 10.
- `System.Net.Http` para HTTP.
- `System.Text.Json` para JSON.
- xUnit para pruebas unitarias.
- Mermaid para el diagrama de clases editable.

7. Ejecución y pruebas

```
cd CooperativaElProgreso  
  
cd ../CooperativaElProgreso.Tests
```